UNIVERSIDAD TECNOLÓGICA ECOTEC



ACTIVIDAD INTEGRADORA 2

Reingeniería, Gestión de Servicios Nativos y Auditoría de Código

NEXO PORTFOLIO

Estudiante: Mike Quishpe Alban
Asignatura: Programación Móvil
Fecha: 28 de octubre de 2026

AVISO INICIAL: Este trabajo utiliza como base el proyecto desarrollado en la Actividad Integradora 1 (portafolio móvil en Flutter), sobre el que se plantean e incorporan mejoras de identidad visual, servicios nativos y documentación.

Documento técnico de reingeniería · Versión 1.0

SECCIÓN I. JUSTIFICACIÓN DEL REBRANDING Y CUADRO COMPARATIVO

1.1 Contexto del proyecto

La aplicación de partida es un portafolio móvil construido con Flutter. La documentación original describe una pantalla principal con presentación, habilidades y proyectos destacados; tarjetas reutilizables; navegación a una pantalla de detalle mediante Navigator; transferencia del objeto Project; y un formulario con validación. La reingeniería conserva ese propósito y añade capacidades propias de una aplicación móvil moderna.

1.2 Nueva propuesta de valor

Nexo Portfolio presenta la experiencia académica y los proyectos técnicos en una interfaz más consistente y, al mismo tiempo, sirve como demostración de integración de hardware y sistema operativo. La conectividad se representa de forma reactiva; el usuario puede seleccionar o capturar imágenes y ajustarlas; y los eventos de prueba producen notificaciones locales. Las funciones están separadas en servicios para reducir el acoplamiento con la interfaz.

1.3 Identidad visual

Elemento	Valor	Uso
Azul noche · #101827	Color primario	Barra superior, icono y superficies de identidad
Turquesa · #20D9D2	Color secundario	Acentos, iconografía y énfasis
Verde lima · #C7F36B	Color de apoyo	Detalles de marca
Fondo · #F5F7FA	Fondo general	Lectura y contraste de tarjetas

Aspecto	Antes (Actividad 1)	Después (Actividad 2)
Nombre comercial	Portafolio personal / ecotec_app	Nexo Portfolio
Icono y splash	Recursos predeterminados de Flutter	Icono monograma N y splash con fondo azul noche
Paleta	Naranja, cian oscuro y gris	Azul noche, turquesa, lima y fondo claro
Identificador Android	com.example.ecotec_app	com.mikequishpe.nexoportfolio
Identificador iOS	com.example.ecotecApp	com.mikequishpe.nexoportfolio
Servicios	Navegación, detalle y formulario	Conectividad, cámara/galería, recorte y notificaciones locales
Persistencia offline	No documentada en el proyecto original	Preferencias locales para borrador, estado de red y ruta de imagen

El cambio de identificador implica que Android/iOS consideran la aplicación una instalación distinta de la anterior. En caso de publicar una actualización sobre la app existente, debe evaluarse si se desea conservar el identificador original.

SECCIÓN II. ARQUITECTURA Y PATRONES DE DESARROLLO

2.1 Arquitectura por responsabilidades

Se propone una separación ligera en capas. Las pantallas se encargan de presentar y recibir acciones; los servicios encapsulan la interacción con plugins del dispositivo; el repositorio concentra el acceso a preferencias locales; y el estado global de conectividad se expone como un stream que actualiza la interfaz. La implementación usa StatefulWidget/setState para este estado concreto y no incorpora Provider/BLoC como dependencia adicional, dado que la aplicación mantiene un alcance pequeño.

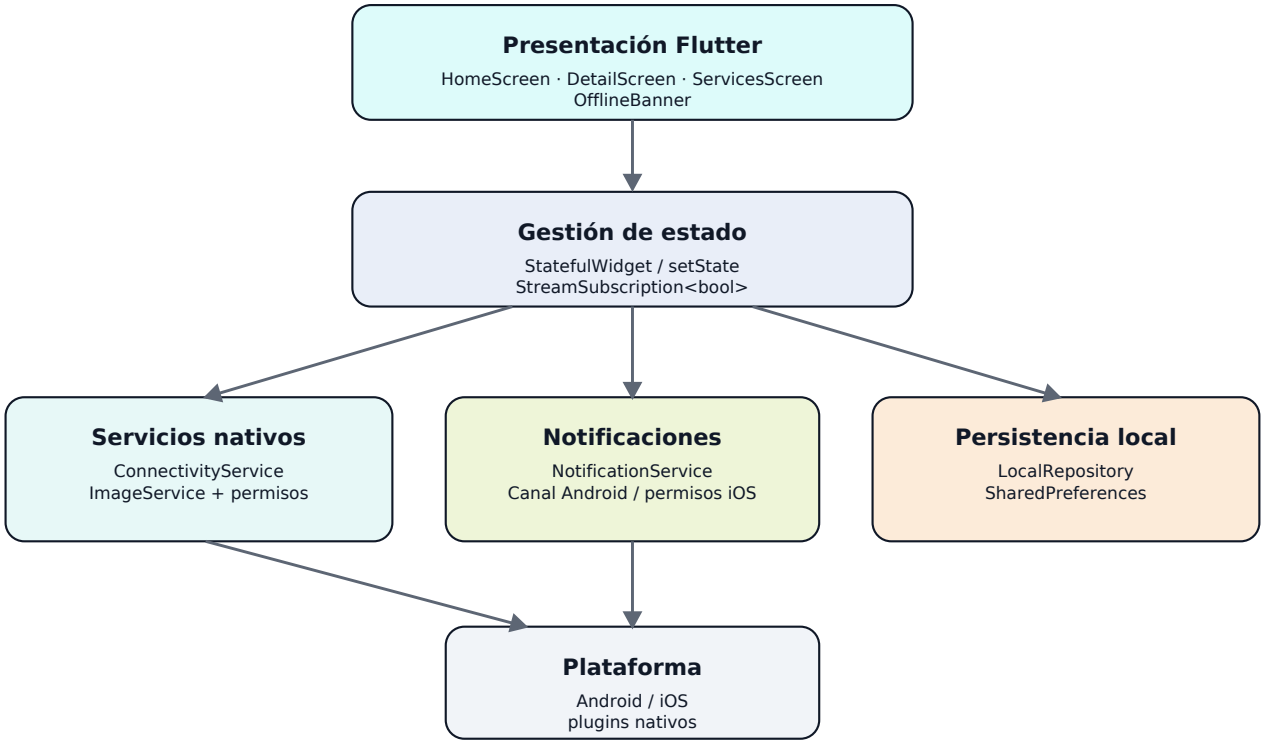


Figura 1. Arquitectura lógica propuesta e incorporada al código. Las flechas representan el flujo de llamadas y resultados; no equivalen a una medición de ejecución.

2.2 Componentes

Componente	Responsabilidad
HomeScreen / DetailScreen	Presentación del portafolio, navegación y validación del formulario.
ServicesScreen	Pantalla de demostración para cámara, galería y notificaciones.
ConnectivityService	Consulta inicial y emisión de cambios de disponibilidad de interfaz de red.
ImageService	Permiso de cámara, selección/captura, recorte y compresión JPEG.
NotificationService	Inicialización del plugin, canal Android y solicitudes de permiso.
LocalRepository	Persistencia con SharedPreferences: borrador, último estado de red y ruta de imagen.
OfflineBanner	Indicador global visible cuando la conectividad reporta que no hay red.

2.3 Decisiones técnicas y limitaciones

La detección con connectivity_plus informa si existe una interfaz de red distinta de none; no garantiza por sí sola que un host de Internet sea accesible. Para una aplicación de producción convendría combinarla con una comprobación de alcance del backend. SharedPreferences es apropiado para pequeños valores y borradores, pero no reemplaza una base de datos o caché de archivos administrada. La ruta de una imagen temporal puede dejar de existir si el sistema limpia el archivo.

SECCIÓN III. EXPLICACIÓN DE CÓDIGO Y MANEJO DE EXCEPCIONES

3.1 Detección reactiva de conectividad

El servicio transforma la lista de estados del plugin en un booleano. distinct() evita notificar repetidamente el mismo estado. La suscripción global actualiza el banner y guarda el último valor conocido; la suscripción se cancela en dispose para evitar fugas de recursos.

```
Stream<bool> get onlineStream => _connectivity.onConnectivityChanged
    .map(_hasNetwork)
    .distinct();

bool _hasNetwork(List<ConnectivityResult> results) =>
    results.any((result) => result != ConnectivityResult.none);
```

Manejo de errores: la consulta inicial y la persistencia están envueltas en try/catch; el stream registra errores mediante onError. La UI sigue mostrando el contenido local si se pierde la red. El banner informa de conectividad de red, no de disponibilidad garantizada de Internet.

3.2 Cámara, galería y permisos

```
if (source == ImageSource.camera) {
  var status = await Permission.camera.status;
  if (status.isDenied || status.isRestricted) {
    status = await Permission.camera.request();
  }
  if (!status.isGranted) {
    throw ImagePermissionException(
      status.isPermanentlyDenied
        ? 'El permiso está bloqueado. Actívalo en Ajustes.'
        : 'Se necesita permiso de cámara.',
      permanentlyDenied: status.isPermanentlyDenied,
    );
  }
}

final selected = await _picker.pickImage(
  source: source, imageQuality: 82,
  maxWidth: 1600, maxHeight: 1600,
);
```

Después de seleccionar o capturar una imagen, ImageCropper presenta la interfaz nativa de recorte y genera una salida JPEG con calidad 82. La selección cancelada devuelve null y no se trata como error. Si el permiso se deniega permanentemente, la interfaz ofrece un acceso a los ajustes del sistema. La galería se abre mediante el selector del sistema, cuya política de permisos varía por versión de Android/iOS.

3.3 Notificaciones locales

```
static const AndroidNotificationChannel _channel =
  AndroidNotificationChannel(
    'nexo_events', 'Eventos de Nexo Portfolio',
    description: 'Avisos locales de acciones realizadas.',
    importance: Importance.high,
  );

await _plugin.show(
  1001, 'Nexo Portfolio',
  'La notificación local está funcionando correctamente.',
  details,
);
```

La inicialización configura ajustes de Android e iOS y registra el canal de Android. Antes de disparar la alerta se solicita permiso. Los fallos se capturan en la pantalla de servicios y se muestran como mensajes legibles. Para probar en segundo plano, se debe minimizar la app y verificar el centro de notificaciones del dispositivo.

3.4 Persistencia y ciclo de vida

LocalRepository centraliza SharedPreferences y guarda el borrador de contacto en JSON, el último estado de red y la ruta de la imagen seleccionada. Los controladores del formulario se liberan en dispose. La suscripción de conectividad también se cancela. La persistencia de una ruta de archivo temporal es una referencia de conveniencia, no una garantía de almacenamiento permanente.

SECCIÓN IV. EVIDENCIAS DE PRUEBAS (QA) Y ENLACE AL REPOSITORIO PÚBLICO

4.1 Repositorio

Repositorio configurado en el historial Git recibido: https://github.com/miquishpe-lang/ACTIVIDAD1_MOVIL. El enlace corresponde al repositorio base. Los cambios de esta entrega están preparados localmente; debe ejecutarse el push desde una sesión autenticada y comprobarse que la rama actualizada sea visible públicamente antes de entregar el informe.

4.2 Pruebas de aceptación

ID	Prueba	Resultado esperado	Estado de evidencia
QA-01	Abrir aplicación	La pantalla de inicio muestra la marca Nexo Portfolio y los proyectos.	Pendiente de ejecutar en emulador/dispositivo
QA-02	Desactivar Wi-Fi/datos	Se muestra OfflineBanner y el contenido estático sigue disponible.	Pendiente de captura real
QA-03	Tomar fotografía	El sistema solicita permiso si corresponde, abre cámara, permite recortar y muestra imagen.	Pendiente de captura real
QA-04	Denegar cámara	La app informa que el permiso es necesario; si es permanente ofrece Ajustes.	Pendiente de captura real
QA-05	Seleccionar de galería	Se abre selector, se permite recorte y se previsualiza imagen.	Pendiente de captura real
QA-06	Enviar notificación	Se solicita permiso y aparece la notificación local.	Pendiente de captura real
QA-07	Borrador de contacto	Los campos se recuperan al volver a abrir el detalle.	Pendiente de ejecutar
QA-08	Análisis y tests	`flutter analyze` y `flutter test` terminan sin errores.	No ejecutado en este entorno

4.3 Capturas que deben agregarse antes de la entrega

Este entorno de elaboración no dispone de Flutter/Android Emulator para ejecutar la aplicación, por lo que no se incluyen capturas de ejecución inventadas. Después de ejecutar `flutter pub get`, `flutter analyze`, `flutter test` y `flutter run`, inserta capturas reales de: (1) inicio con nuevo branding, (2) banner offline tras desactivar la red, (3) diálogo/permisos de cámara, (4) imagen recortada y previsualizada, y (5) notificación en el centro de notificaciones. Las capturas deben mostrar el dispositivo/emulador y no contener datos privados.

4.4 Auditoría Git

La carpeta de trabajo incluye el repositorio Git original. Para cumplir la rúbrica, comprueba que existan al menos cinco commits relevantes, ramas main/develop/feature y que la versión actual esté publicada. Los commits locales no equivalen a commits publicados. En GitHub, revisa el historial de commits y la lista de ramas después de ejecutar el push.

SECCIÓN V. CONCLUSIONES Y LECCIONES APRENDIDAS

La reingeniería muestra cómo una aplicación móvil puede evolucionar sin desechar las funcionalidades de la primera entrega. El rebranding se centraliza en el tema y en los recursos nativos, mientras que conectividad, imágenes y notificaciones se aíslan en servicios específicos. Esta separación facilita el mantenimiento, permite cambiar plugins con menor impacto y hace más clara la responsabilidad de cada componente.

La gestión del ciclo de vida es esencial: los streams deben cancelarse, los controladores deben liberarse y las operaciones asíncronas deben comprobar que el widget continúa montado antes de actualizar la interfaz. Los permisos no se pueden asumir; el sistema puede denegarlos o bloquearlos, por lo que el flujo debe contemplar mensajes y recuperación desde Ajustes.

La conectividad es un indicador de estado de red y no una prueba absoluta de acceso a un servicio remoto. En una solución productiva se debería verificar el endpoint de negocio y definir políticas de reintento, caducidad y sincronización. La persistencia local ayuda a conservar borradores y valores pequeños, pero las imágenes requieren una estrategia de almacenamiento duradero si deben sobrevivir a la limpieza del sistema.

Por último, el control de versiones permite rastrear cambios, revisar la calidad y separar el desarrollo de la versión estable. La evidencia definitiva requiere completar las pruebas manuales, ejecutar análisis y pruebas automatizadas con el SDK del estudiante y publicar las ramas en el repositorio público. No se debe presentar una prueba como exitosa si no se ha ejecutado y observado.

Checklist final antes de entregar

[ ] Ejecutar flutter pub get, flutter analyze y flutter test.

[ ] Probar en Android físico/emulador cámara, galería, offline y notificaciones.

[ ] Tomar e insertar las capturas reales en esta sección de evidencias.

[ ] Publicar commits y ramas en GitHub/GitLab y confirmar que el repositorio sea público.

[ ] Verificar la URL del repositorio y actualizar el enlace si cambia.

Anexo técnico: archivos principales modificados

`lib/main.dart`; `lib/services/connectivity_service.dart`; `lib/services/image_service.dart`; `lib/services/notification_service.dart`; `lib/repositories/local_repository.dart`; `lib/screens/services_screen.dart`; `lib/widgets/offline_banner.dart`; `lib/screens/detail_screen.dart`; `pubspec.yaml`; `android/app/src/main/AndroidManifest.xml`; `android/app/build.gradle.kts`; `ios/Runner/Info.plist`; recursos de icono y splash; `README.md`.